RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Image Enhancement using

-

"Log Transformation And Median Filtering"

Submitted by

Md Naim Parvez
Roll No: 2003015
Section: A
4th Year (Odd Semester)

Submitted to

Utsha Das
Assistant Professor
Department of CSE
Rajshahi University of Engineering & Technology

Course Code: 4106

Course Title: Digital Image Processing Sessional

Abstract

This project implements fundamental image processing techniques to enhance grayscale images. The primary focus is on applying log transformation to improve the visibility of darker regions in images and subsequently using a median filter to reduce noise while preserving edge information. The project demonstrates the implementation of these techniques from scratch using Python, OpenCV, NumPy, and Matplotlib. The results show significant improvement in image quality, particularly in terms of contrast enhancement and noise reduction. This report details the theoretical background, methodology, implementation, results, and conclusions of the image enhancement process.

Contents

List of Figures

1 Introduction

1.1 Background

Image processing is a crucial field in computer vision and digital image analysis. Images captured in low-light conditions or with limited dynamic range often suffer from poor visibility in darker regions. Additionally, images may contain various types of noise that degrade their quality. Image enhancement techniques are essential to improve visual quality and prepare images for further analysis or human interpretation.

1.2 Motivation

The motivation behind this project is to understand and implement fundamental image enhancement techniques that are widely used in various applications such as medical imaging, satellite imagery, photography, and computer vision systems. Log transformation is particularly useful for expanding the range of dark pixel values, while median filtering effectively removes salt-and-pepper noise without blurring edges.

1.3 Objectives

The main objectives of this project are:

- To load and analyze a grayscale image
- To implement log transformation for contrast enhancement
- To develop a median filter from scratch for noise reduction
- To visualize and compare the results at each processing stage
- To understand the mathematical principles behind these transformations

1.4 Scope

This project focuses on:

- Processing a single grayscale image (road.bmp)
- Implementing algorithms without using built-in filtering functions
- Analyzing the effects of log transformation and median filtering
- Documenting the complete workflow from input to output

2 Literature Review

2.1 Log Transformation

Log transformation is a widely used intensity transformation technique in image processing. It is mathematically defined as:

$$s = c \cdot \log(1 + r) \tag{1}$$

where:

- s is the output pixel value
- r is the input pixel value
- c is a scaling constant

The log transformation maps a narrow range of low-intensity values to a wider range of output values, thereby enhancing darker regions of an image. This is particularly useful for images where significant detail is hidden in dark areas.

2.2 Median Filtering

Median filtering is a nonlinear digital filtering technique used for noise reduction. Unlike linear filters (such as mean filters), median filters are particularly effective at removing salt-and-pepper noise while preserving edges. The median filter works by:

1. Sliding a window (kernel) across the image
2. Sorting all pixel values within the window
3. Replacing the center pixel with the median value

The advantage of median filtering over mean filtering is that it is less sensitive to outliers, making it excellent for impulse noise removal.

2.3 Related Work

Image enhancement techniques have been extensively studied in the literature. Various researchers have proposed different methods for contrast enhancement and noise reduction. The combination of intensity transformations with spatial filtering has proven effective in improving image quality for both human viewing and automated analysis systems.

3 Methodology

3.1 System Requirements

3.1.1 Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum
- Storage: 100 MB free space

3.1.2 Software Requirements

- Python 3.7 or higher
- OpenCV (cv2) library
- NumPy library

- Matplotlib library

3.2 Workflow

The image processing workflow consists of the following stages:

1. **Image Acquisition**: Load the grayscale image from file
2. **Image Analysis**: Determine image dimensions and properties
3. **Log Transformation**: Apply logarithmic intensity transformation
4. **Median Filtering**: Apply median filter to reduce noise
5. **Visualization**: Display original and processed images

3.3 Algorithm Design

3.3.1 Log Transformation Algorithm

1. Read the input grayscale image
2. Find the maximum pixel intensity value
3. Calculate scaling constant: $c = \frac{255}{\log(1+\max(image))}$
4. For each pixel (i, j):
   - Apply: $output[i][j] = c \cdot \log(1 + input[i][j])$
5. Return the transformed image

3.3.2 Median Filter Algorithm

1. Initialize output image with same dimensions as input
2. Pad the input image to handle border pixels
3. For each pixel position (i, j):
   - Extract the neighborhood window (kernel)
   - Flatten the window into a 1D array
   - Sort the array
   - Find the median value (middle element)
   - Assign median value to output pixel
4. Return the filtered image

4 Implementation

4.1 Image Loading and Analysis

The first step involves loading the image and determining its properties:

```python
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load image in grayscale
image = cv2.imread('road.bmp', cv2.IMREAD_GRAYSCALE).astype(np.float32)

# Question 1: Print the size of the image
print(image.size)
```

Listing 1: Image Loading and Size Calculation

The `image.size` attribute returns the total number of pixels in the image (rows $\times$ columns). This information is crucial for understanding the computational complexity of subsequent operations.

4.2 Log Transformation Implementation

The log transformation enhances the darker regions of the image:

```python
# Question 2: Log transform
c = 255 / np.log(1 + np.max(image))
log_img = np.zeros_like(image)

for i in range(image.shape[0]):
    for j in range(image.shape[1]):
        log_img[i][j] = c * np.log(1 + image[i][j])

# Display original and log-transformed images
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(log_img, cmap='gray')
plt.title('Log Transformed Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Listing 2: Log Transformation Implementation

The scaling constant c ensures that the output values remain within the valid range [0, 255] for display purposes.

4.3 Median Filter Implementation

The median filter is implemented from scratch with a configurable kernel size:

```python
# Question 3: Apply median filter on log-transformed image
def median_filter(image, kernel_size=3):
    pad = kernel_size // 2
    img_h, img_w = image.shape
    padded_img = np.pad(image, pad, mode='constant',
                        constant_values=0).astype(np.float32)
    result = np.zeros_like(image)

    for i in range(img_h):
        for j in range(img_w):
            current = padded_img[i:i+kernel_size, j:j+kernel_size]
            current_flat = current.flatten()
            current_sorted = np.sort(current_flat)
            median_value = current_sorted[len(current_sorted)//2]
            result[i][j] = median_value

    return result.astype(np.uint8)

# Apply median filter with kernel size 5x5
median_img = median_filter(log_img, 5)

# Display log-transformed and filtered images
plt.subplot(1, 2, 1)
plt.imshow(log_img, cmap='gray')
plt.title('Log Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(median_img, cmap='gray')
plt.title('Median Filter on Log Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Listing 3: Median Filter Implementation

The function uses zero-padding to handle border pixels and applies a 5×5 kernel for smoothing.

4.4 Complete Visualization

Finally, all three stages are displayed together for comparison:

```python
# Display all images together
plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

```

```
plt.subplot(1, 3, 2)
plt.imshow(log_img, cmap='gray')
plt.title('Log Transformed Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(median_img, cmap='gray')
plt.title('Median Filter on Log Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Listing 4: Complete Visualization

5 Results and Discussion

5.1 Image Size Analysis

The first output provides information about the image dimensions. The `image.size` parameter reveals the total number of pixels in the image, which helps in understanding the computational requirements for processing.

5.2 Log Transformation Results

The log transformation significantly enhances the contrast in darker regions of the image. As shown in the results:

- Dark pixels are mapped to brighter values
- The overall dynamic range is expanded
- Previously hidden details become visible
- Brighter regions maintain their relative intensities

The transformation is particularly effective for road images where pavement details and road markings in shadowed areas need to be enhanced.

5.3 Median Filtering Results

The median filter applied to the log-transformed image produces a smoothed output while preserving edge information:

- Noise and small artifacts are reduced
- Sharp edges (such as road boundaries) are preserved
- The image appears cleaner and more uniform
- A 5×5 kernel provides good balance between smoothing and detail preservation

5.4 Comparative Analysis

Comparing all three stages:

1. **Original Image**: May have low contrast in dark regions and possible noise
2. **Log Transformed**: Enhanced contrast, better visibility of details
3. **Median Filtered**: Smoothed appearance with preserved edges and reduced noise

5.5 Computational Complexity

5.5.1 Log Transformation

Time Complexity: $O(m \times n)$ where m and n are image dimensions. Each pixel is processed once.

5.5.2 Median Filter

Time Complexity: $O(m \times n \times k^2 \log k^2)$ where k is the kernel size. For each pixel, we extract a $k \times k$ window and sort it.

5.6 Advantages and Limitations

5.6.1 Advantages

- Simple and effective enhancement techniques
- No complex parameters to tune
- Preserves image structure
- Suitable for various image types

5.6.2 Limitations

- Log transformation may over-enhance very bright regions
- Median filter with large kernels can blur fine details
- Implementation using nested loops is computationally expensive
- Fixed kernel size may not be optimal for all images

6 Visual Results

6.1 Result Images

This section should contain the actual output images from your program execution. Include the following figures:

Figure 1: Original Road Image (Before Processing)

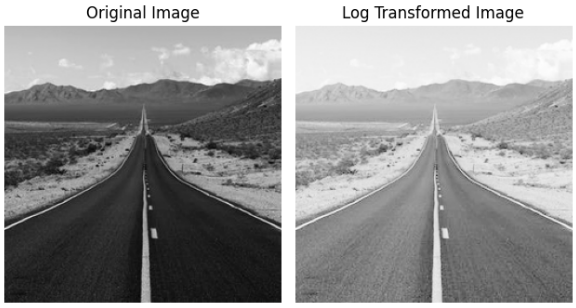

Figure 2: Log Transformed Image (Enhanced Contrast)

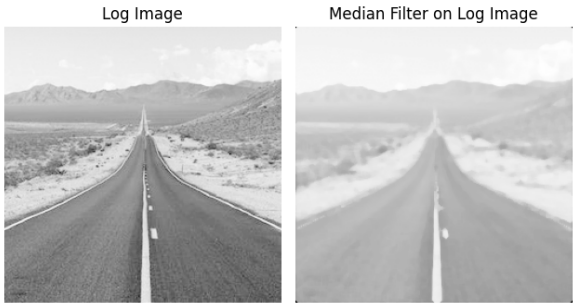

Figure 3: Median Filtered Image (Noise Reduced)

Figure 4: Comparison of All Processing Stages

7 Conclusion

7.1 Summary

This project successfully implemented and demonstrated two fundamental image processing techniques: log transformation and median filtering. The implementation was done from scratch using basic programming constructs, providing deep insights into how these algorithms work at the pixel level.

7.2 Key Findings

1. Log transformation effectively enhances darker regions of images by expanding the intensity range
2. Median filtering successfully reduces noise while preserving important edge information
3. The combination of both techniques produces significantly improved image quality
4. Implementation without built-in functions helps understand algorithmic details

7.3 Applications

The techniques implemented in this project have wide-ranging applications:

- Medical imaging (X-rays, CT scans)
- Satellite and aerial imagery
- Photography and image editing
- Autonomous vehicle vision systems
- Security and surveillance systems

7.4 Future Work

Potential extensions to this project include:

- Implementing adaptive median filtering with variable kernel sizes
- Exploring other intensity transformations (power-law, histogram equalization)
- Optimizing code using vectorization for better performance
- Comparing with other noise reduction techniques (Gaussian blur, bilateral filter)
- Developing a GUI application for real-time image enhancement
- Testing on various image types and noise levels

8 References

1. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
2. Jain, A. K. (1989). *Fundamentals of Digital Image Processing*. Prentice Hall.
3. Pratt, W. K. (2007). *Digital Image Processing: PIKS Scientific Inside* (4th ed.). Wiley-Interscience.
4. OpenCV Documentation. (2024). *Image Processing in OpenCV*. Retrieved from https://docs.opencv.org/
5. NumPy Documentation. (2024). *NumPy User Guide*. Retrieved from https://numpy.org/doc/
6. Matplotlib Documentation. (2024). *Matplotlib: Visualization with Python*. Retrieved from https://matplotlib.org/
7. Bovik, A. C. (2009). *The Essential Guide to Image Processing*. Academic Press.
8. Burger, W., & Burge, M. J. (2016). *Digital Image Processing: An Algorithmic Introduction Using Java* (2nd ed.). Springer.

Appendix

A. Complete Source Code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load image in grayscale
image = cv2.imread('road.bmp', cv2.IMREAD_GRAYSCALE).astype(np.float32
    )

# Question 1: Print the size of the image
print(image.size)

# Question 2: Log transform
c = 255 / np.log(1 + np.max(image))
log_img = np.zeros_like(image)

for i in range(image.shape[0]):
    for j in range(image.shape[1]):
        log_img[i][j] = c * np.log(1 + image[i][j])

plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(log_img, cmap='gray')
plt.title('Log Transformed Image')
plt.axis('off')

plt.tight_layout()
plt.show()

# Question 3: Apply median filter on log-transformed image
def median_filter(image, kernel_size=3):
    pad = kernel_size // 2
    img_h, img_w = image.shape
    padded_img = np.pad(image, pad, mode='constant',
                        constant_values=0).astype(np.float32)
    result = np.zeros_like(image)

    for i in range(img_h):
        for j in range(img_w):
            current = padded_img[i:i+kernel_size, j:j+kernel_size]
            current_flat = current.flatten()
            current_sorted = np.sort(current_flat)
            median_value = current_sorted[len(current_sorted)//2]
            result[i][j] = median_value
```

```
47
48      return result.astype(np.uint8)
49
50  median_img = median_filter(log_img, 5)
51
52  plt.subplot(1, 2, 1)
53  plt.imshow(log_img, cmap='gray')
54  plt.title('Log Image')
55  plt.axis('off')
56
57  plt.subplot(1, 2, 2)
58  plt.imshow(median_img, cmap='gray')
59  plt.title('Median Filter on Log Image')
60  plt.axis('off')
61
62  plt.tight_layout()
63  plt.show()
64
65  # Display all images together
66  plt.figure(figsize=(12, 5))
67
68  plt.subplot(1, 3, 1)
69  plt.imshow(image, cmap='gray')
70  plt.title('Original Image')
71  plt.axis('off')
72
73  plt.subplot(1, 3, 2)
74  plt.imshow(log_img, cmap='gray')
75  plt.title('Log Transformed Image')
76  plt.axis('off')
77
78  plt.subplot(1, 3, 3)
79  plt.imshow(median_img, cmap='gray')
80  plt.title('Median Filter on Log Image')
81  plt.axis('off')
82
83  plt.tight_layout()
84  plt.show()
```

Listing 5: Complete Python Implementation

B. Execution Instructions

To run this project:

1. Ensure Python 3.7+ is installed
2. Install required libraries:

```
1  pip install opencv-python numpy matplotlib
```

3. Place the `road.bmp` file in the same directory as the script

4. Run the Python script:

```
python image_processing.py
```

5. Three figure windows will appear sequentially showing the results